Università di Genova

Embedded Systems

Introduction

Prof. Enrico Simetti

What is an Embedded System?

- “An embedded system is a special-purpose computer system designed to perform one or a few dedicated functions, often with real-time computing constraints. It is usually embedded as part of a complete device including hardware and mechanical parts.” (Wikipedia)
- An embedded processor is a ”specialized processor that is part of a larger system and is targeted for a specific application” (Katz, Gentile, Embedded Media Processing)
- “Embedded systems are computing systems with tightly coupled hardware and software integration, that are designed to perform a dedicated function” (Li, Yao, Real-Time Concepts for Embedded Systems)

What is an Embedded System?

Embedded System: A specialized computing system designed to perform dedicated functions within a larger system. It integrates hardware and software, often operating in real-time with constraints on power, memory, and processing. Common in robotics, automotive, and IoT applications. (ChatGPT)

What do “embedded” and “system” refer to?

The expression embedded system almost always refers to an electronic architecture (system) used for controlling the behaviour of the entity encapsulating (embedding) it.

What does “controlling the behaviour” mean?

Controlling a system here simply means making it work as expected. It might require different functionalities:

- acquiring data from the environment
- performing computations
- communicating with other systems
- driving motors or other actuators
- reacting to external events or internal conditions
- interacting with users.

In other words, executing actions of whatsoever nature

Where can I find an embedded system?

Embedded systems span all aspects of modern life and there are many examples of their use:

- consumer electronics (mobile phones, digital cameras, GPS receivers, printers, etc.)
- household appliances (microwave ovens, dishwashers, washing machines, coffee makers, thermostats, home automating devices, etc.)
- transportation (airplane avionics, automotive power management and safety systems, etc.)
- medical equipment (devices for medical imaging, vital signs monitoring, non-invasive inspection, …)
- smart sensors for: environmental monitoring, water quality monitoring, fires detection, etc.
- motor controllers for several applications
- robotic systems
- …

Embedded Systems vs Computers

- Which is the main difference among an embedded system and a traditional computer?

Embedded systems are specialized systems: They do just what is required by the target system and nothing else. They are application-oriented devices.

Conventional computers can do more things: playing an MP3 file, editing a document, browsing internet or just making computations. They are general-purpose devices.

- Why not using a general-purpose computer as an embedded platform?

The main answer is cost. Embedded system are very cost-sensitive devices. They are part of a commercial good which has to be sold on the market at the lowest possible price.

- Does it mean that an embedded system is “weaker” than a computer?

Absolutely not! The attention to costs simply pushes for realizing dedicated architectures, as much as possible optimized for performing the required tasks and nothing else.

What does the optimizing effort imply?

It mainly means limited hardware resources (processing power, memory, peripherals). Less components means:

- lower cost
- higher reliability
- smaller size
- reduced power consumption
- less produced heat

Why should I care of all such issues?

- Almost always embedded systems have to be small-size devices.
- Sometimes they have to work in difficult environmental conditions.
- Very often they have to be portable devices.
- Always they have to be reliable devices, with on-line failure-detection and management capabilities without assuming human intervention

Specific software skills are required

close hardware interaction:

- device configuration and programming
- peripherals configuration and programming
- dynamic power management

code “optimized” for the device at hands

- few bit architectures
- fixed point arithmetic
- compiler specific data structures and instructions

real-time programming:

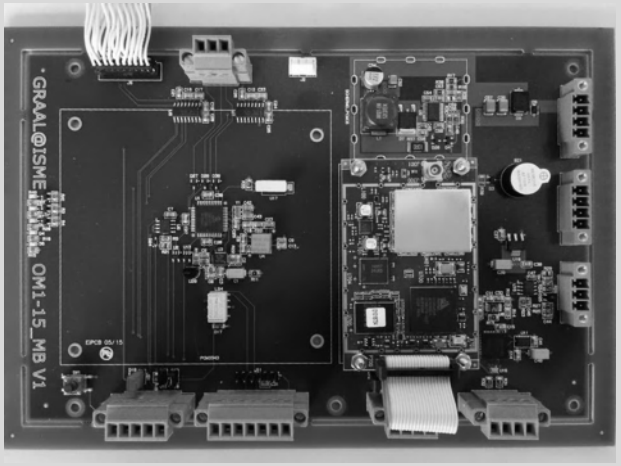
- event-driven code
- time-sensitive or time-critical applications

more issues related with:

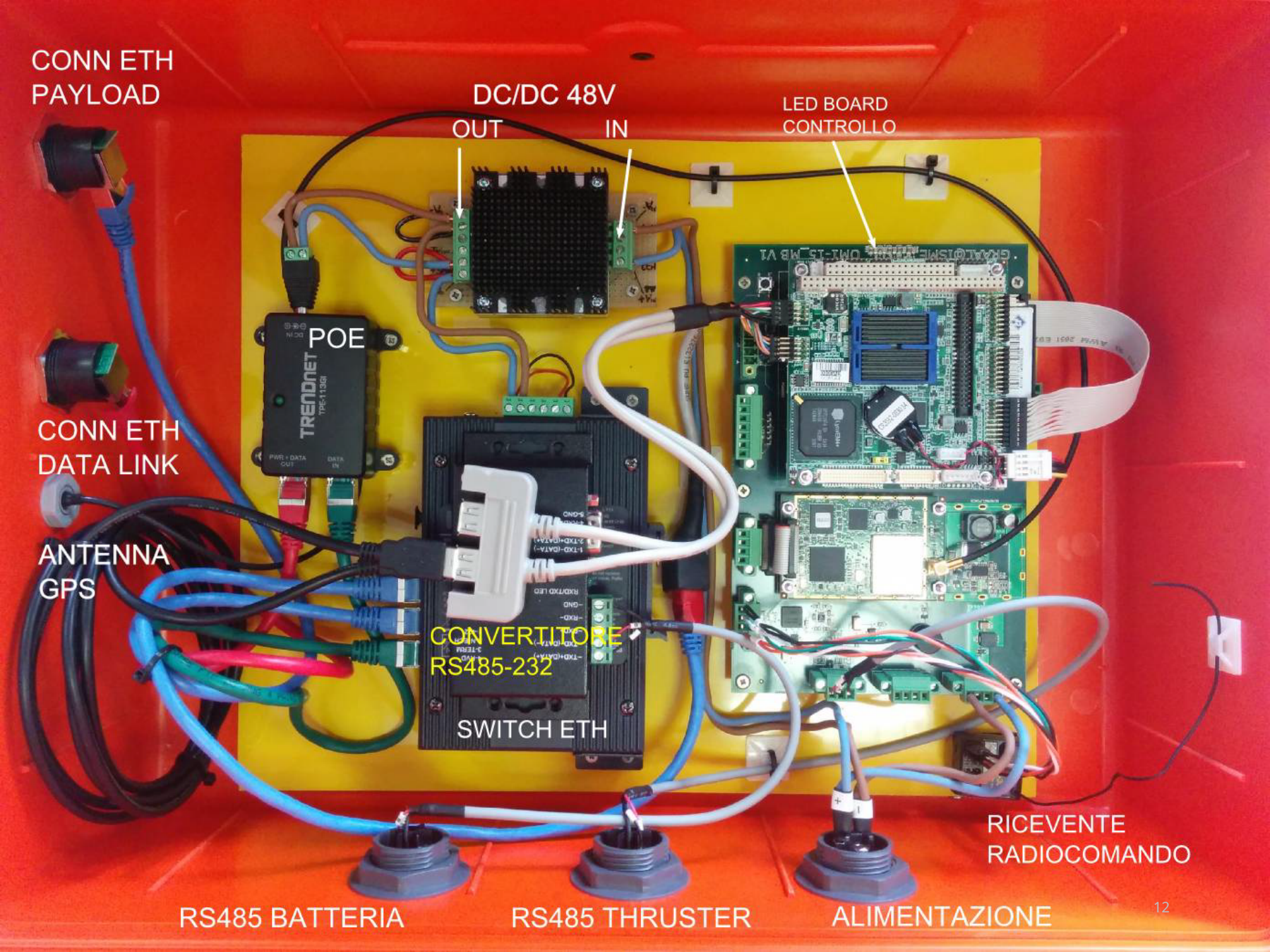
- code debugging
- device programming
- application testing
- dedicated development tools

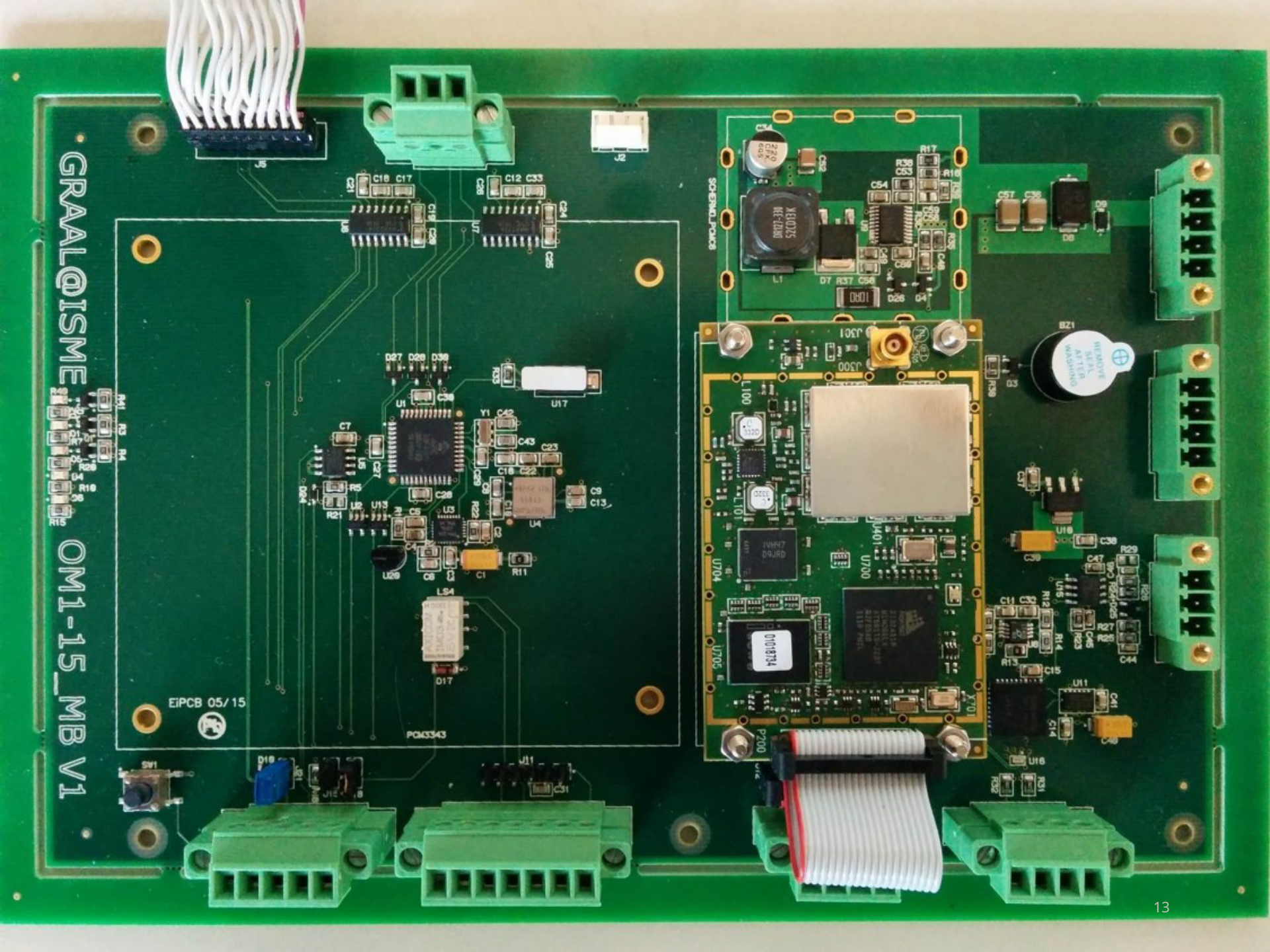
Università di Genova

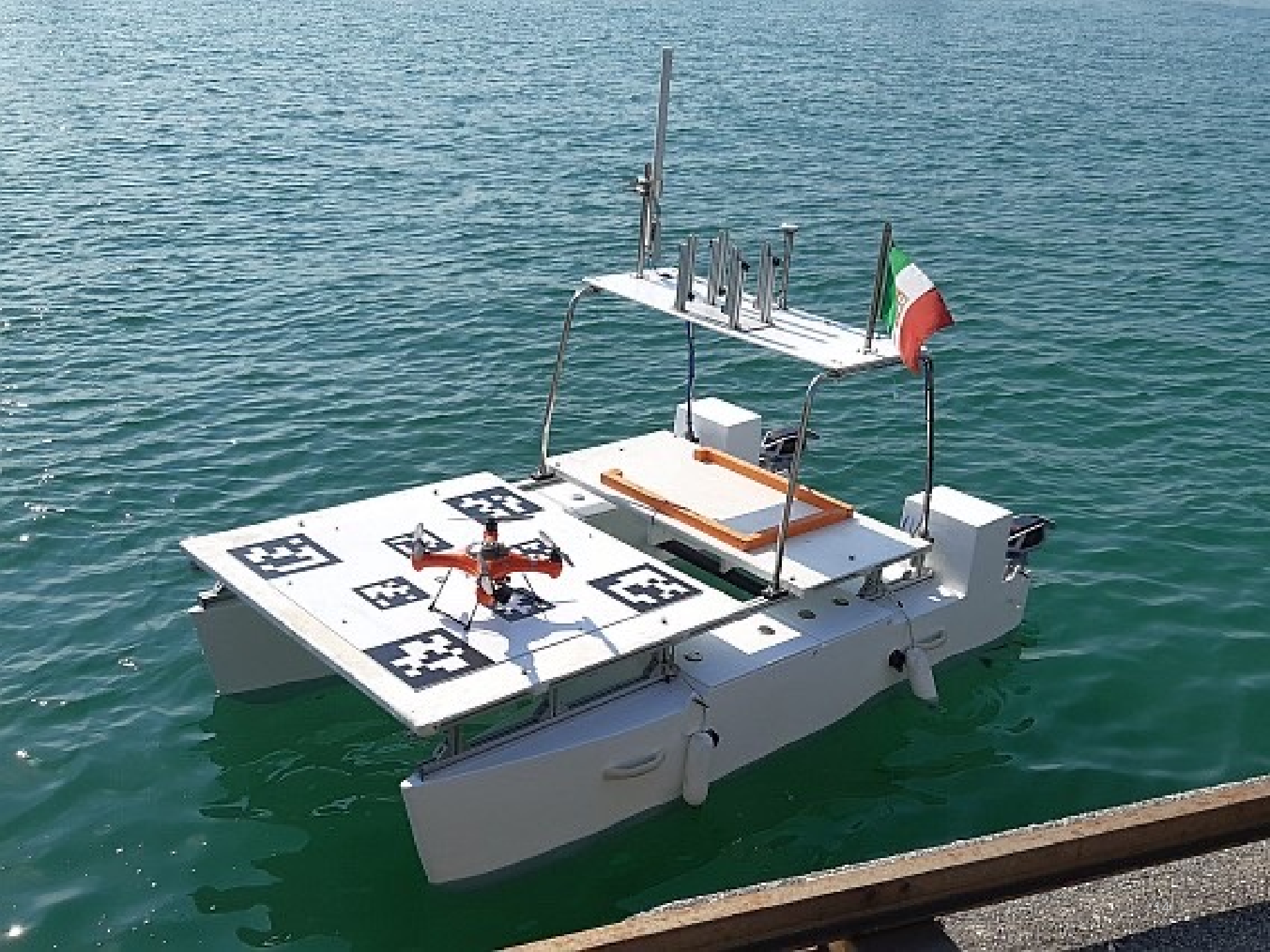
Example of Embedded Systems

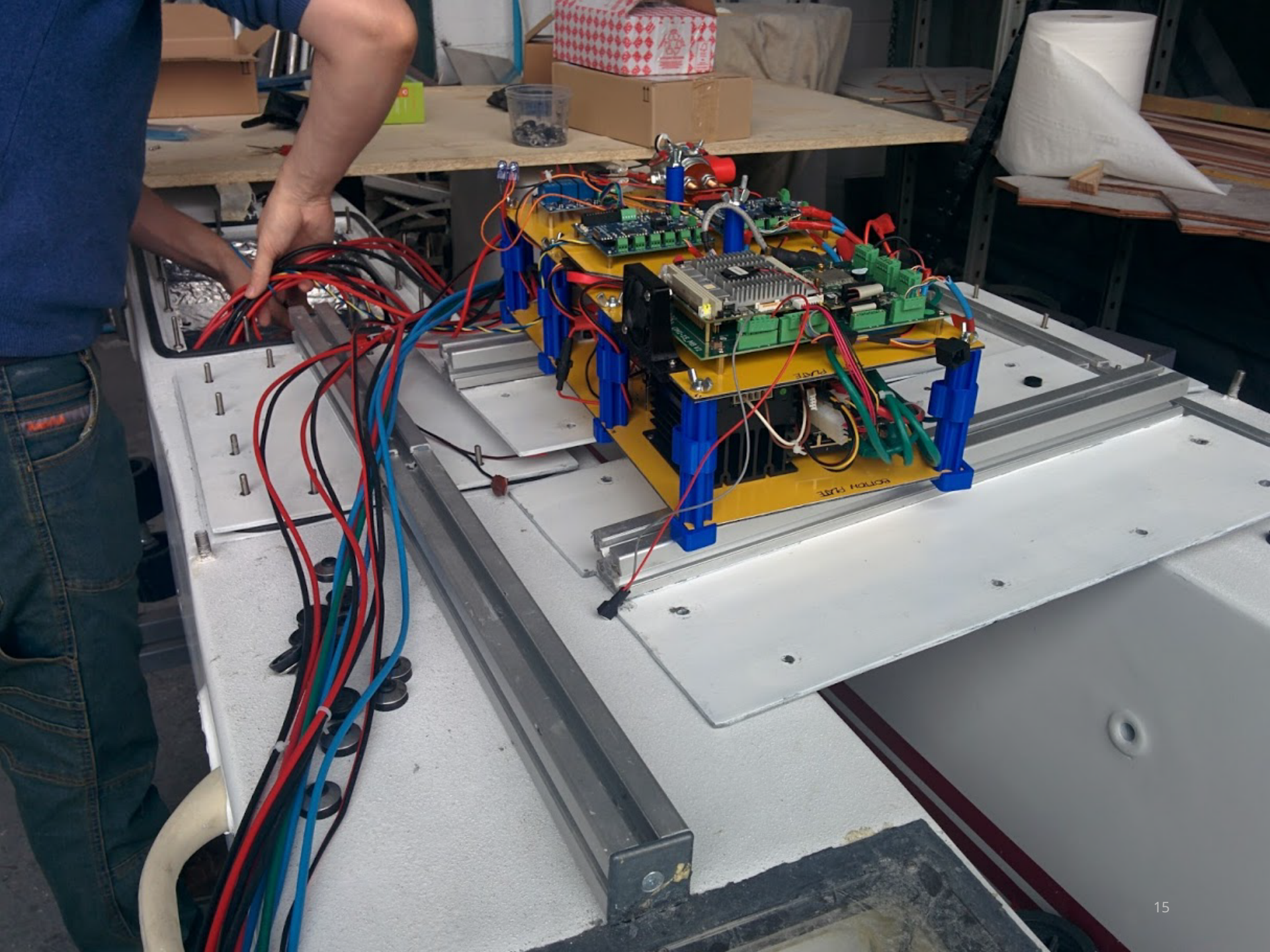


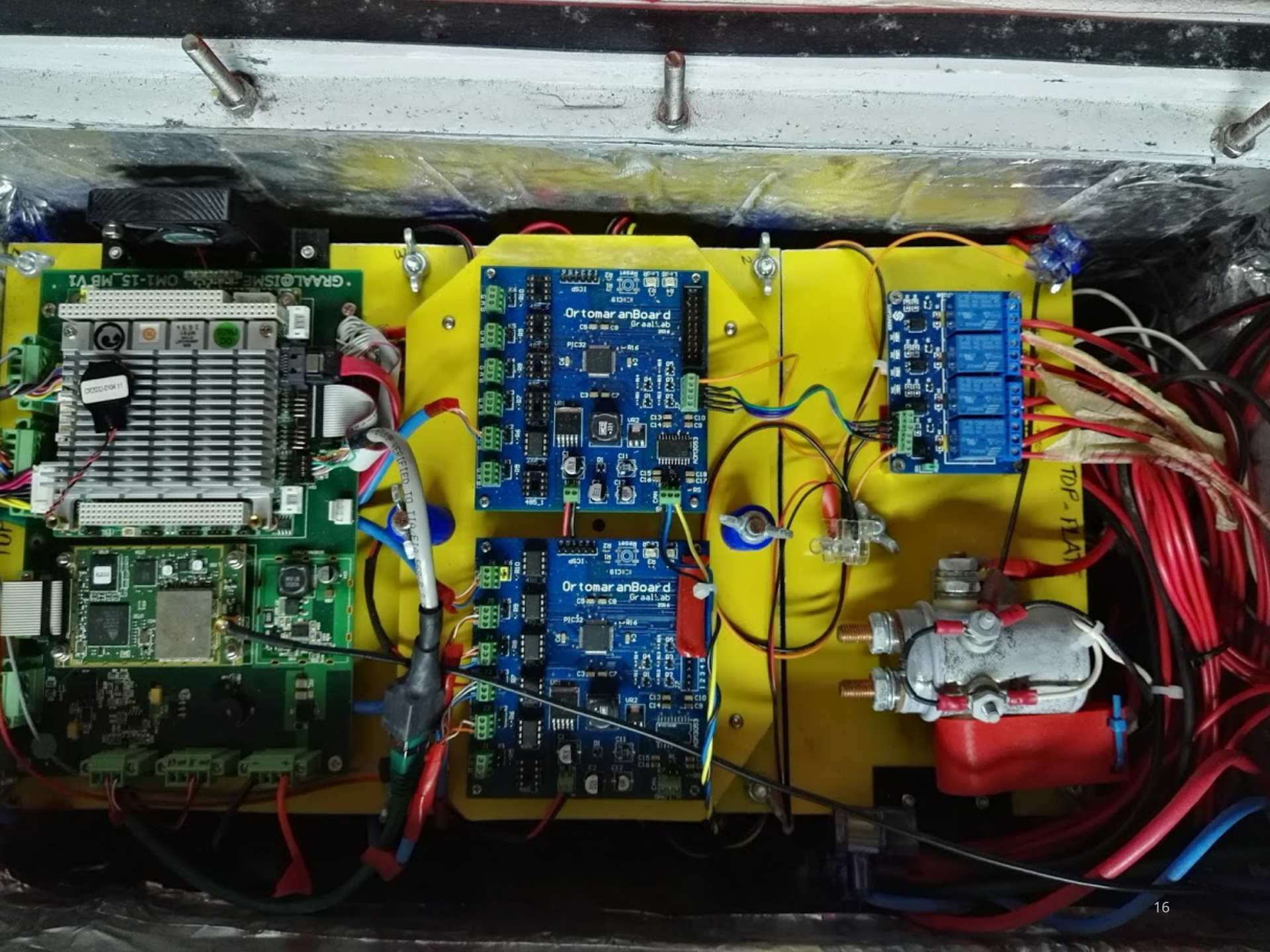



CONN ETH
PAYLOAD
DC/DC 48V
OUT
IN
LED BOARD
CONTROLLO
POE
CONN ETH
DATA LINK
ANTENNA
GPS
CONVERTITORE
RS485-232
SWITCH ETH
RICEVENTE
RADIOCOMANDO
RS485 BATTERIA
RS485 THRUSTER
ALIMENTAZIONE


GRAAL@ISME OM1-15_MB V1
EiPCB 05/15
PCB3343

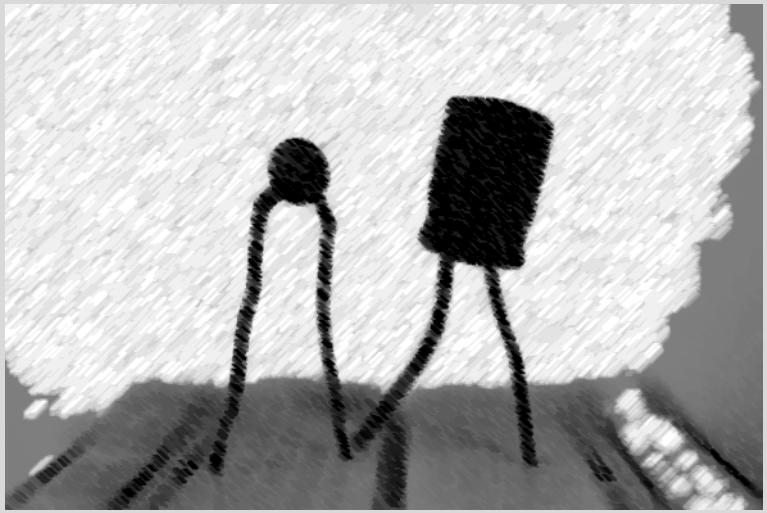





OrtomaranBoard
Graal!ab
PIC32
OrtomaranBoard
Graal!ab
PIC32

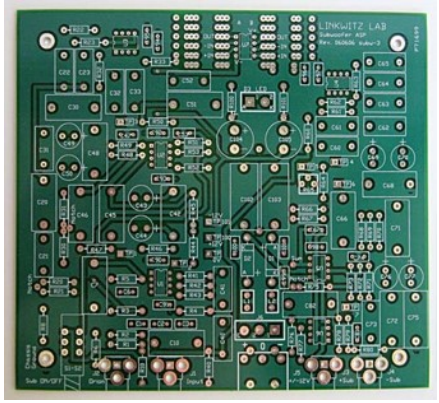
Università di Genova

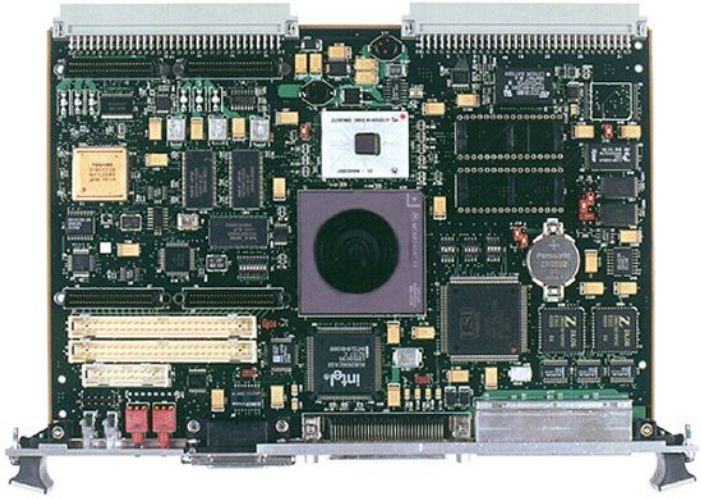
Hardware basics



Printed Circuit Board

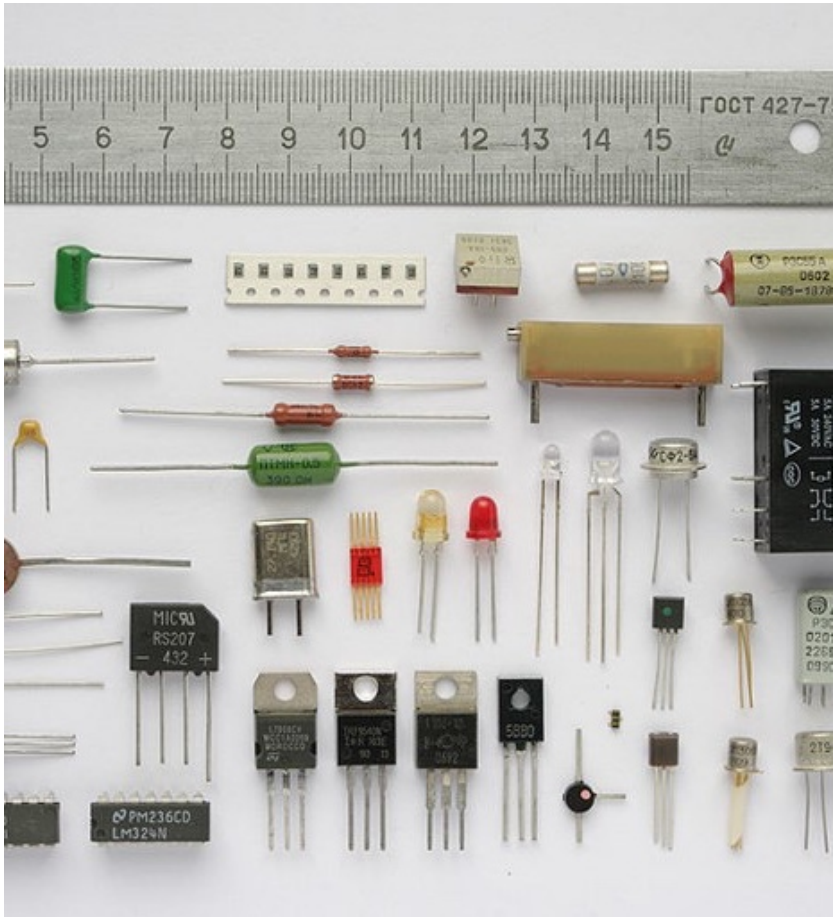
PCB (Printed Circuit Board): Board used for realizing electronic circuits. It is made of dielectric layers laminated with epoxy resins and provides conductive paths and mechanical support to components.





Components on a PCB

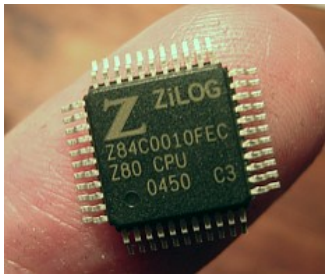
Electronic Components: Basic electronic elements of various nature, constituting electronic circuits, packaged in a discrete form with two or more connecting pads (or pins)



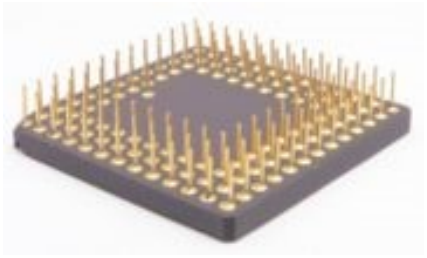
Integrated Circuits

Integrated Circuit (CHIP): Miniaturized electronic circuit contained inside a single component.









General characteristics are:

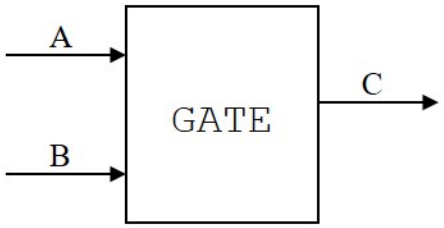
- nature: analog, digital, mixed
- package: SIP (Single-In-line Package), DIP (Dual-In-line Package), QFP (Quad Flat Package), PGA (Pin Grid Array)
- mounting technique: through holes, sockets, SMD (Sourface Mounting Devices)

Logic Gates

Logic Gate: The basic building block of any processing device implementing a logic function (AND, OR, NOT, …)

Main features:

- can be implemented with different transistor technologies (mostly CMOS or TTL).
- have at least one (usually more) input and one output
- input and output values are the logical values true ‘1’ and false ‘0’
- have no memory (i.e. the output depends only on the current value of the inputs)
- can be described through a truth table. If n is the number of inputs of a logic gate, its truth table has $2n$ rows and $n+1$ columns

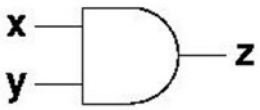


A	B	C
0	0	0/1
0	1	0/1
1	0	0/1
1	1	0/1

Logic Gates

AND GATE: The output value is 1 if and only if all the inputs are 1.

Otherwise, the output value is 0.

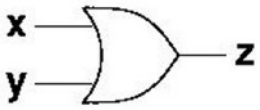


z=xy

x	y	z
0	0	0
0	1	0
1	0	0
1	1	1

OR GATE: The output value is 1 if at least one of the inputs are 1.

Otherwise, the output value is 0.

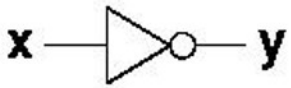


z=x+y

x	y	z
0	0	0
0	1	1
1	0	1
1	1	1

Logic Gates

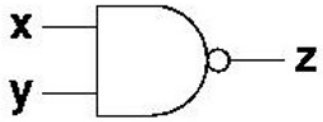
NOT GATE: It has only one input. The output value is 1 if and the input is 0. Otherwise, it is 0.



$y=\overline{x}$

x	y
0	1
1	0

NAND GATE: The NAND-gate is an AND-gate with an inverter on the output. Its output is 0 if all its inputs are 1. Otherwise, it is 1

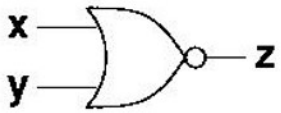


$z=\overline{xy}$

x	y	z
0	0	1
0	1	1
1	0	1
1	1	0

Logic Gates

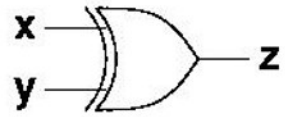
NOR GATE: The output value is 1 if and only if all the inputs are 0. Otherwise, the output value is 0.



$$z=\overline{x+y}$$

x	y	z
0	0	1
0	1	0
1	0	0
1	1	0

XOR GATE: The output value is 1 if and only if exactly one input is 1. Otherwise, it is 0.



$$z=x \oplus y$$

x	y	z
0	0	0
0	1	1
1	0	1
1	1	0

Combinatorial Circuits

Combinatorial circuit: a generalization of the concept of gate, implementing a specific multi-input multi-output logic function

Main features:

- it is made of basic gates
- it has no memory
- can be described through a truth table. If *n* is the number of inputs and *m* the number of outputs, the truth table has 2^n rows and *n*+*m* columns
- for a given logical function, different alternative combinatorial circuit can be obtained (by varying the number and/or the kind of basic gates employed)

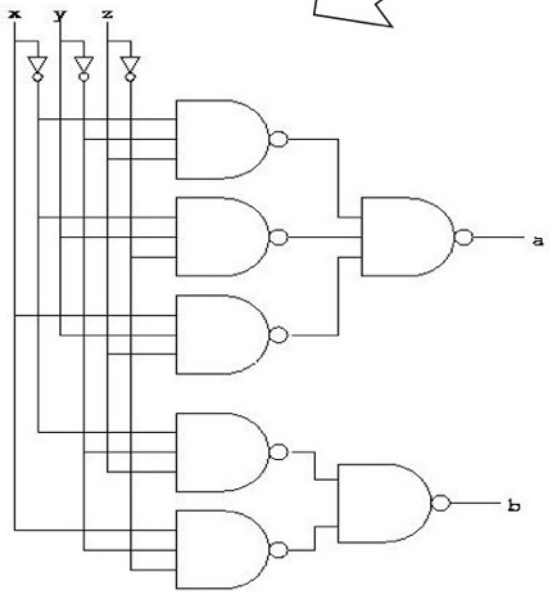
In designing a combinatorial circuit typically, one wants to minimize one or more of:

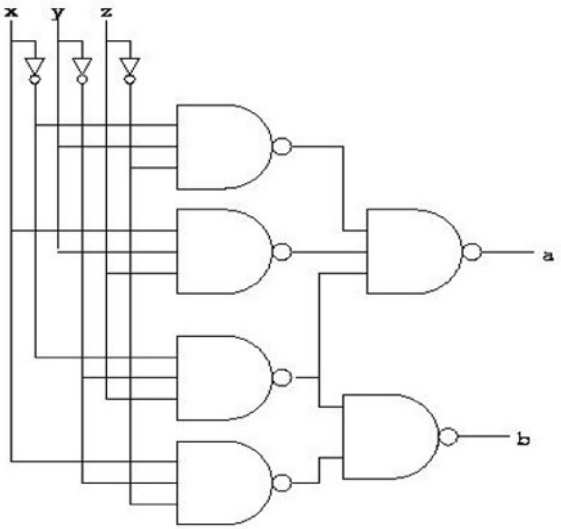
- the number of used gates
- the maximum time a signal takes to traverse the overall circuit (i.e., the propagation time)
- the maximum number of gates an input has to traverse before reaching an output (i.e., the number of layers of the circuit)

Combinatorial Circuits

Example: two equivalent combinatorial circuits

x y z	a b
0 0 0	0 0
0 0 1	1 1
0 1 0	1 0
0 1 1	0 0
1 0 0	0 1
1 0 1	0 0
1 1 0	0 0
1 1 1	1 0





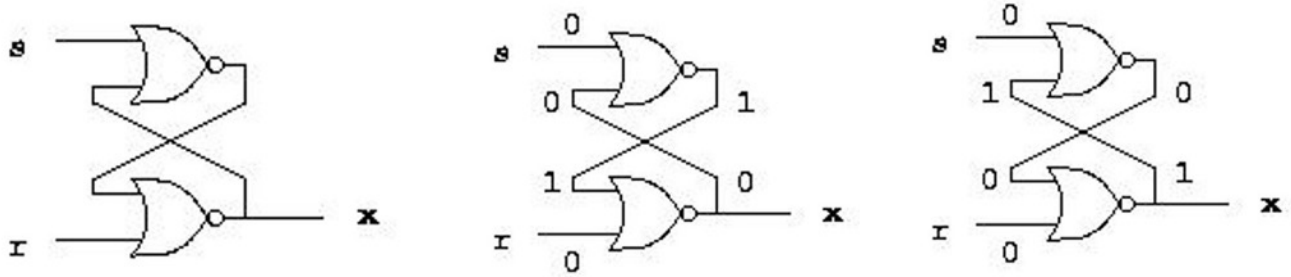
Basic memory elements: latch and flip-flop

Latch, Flip-flop: circuit elements whose output depends not only on the current inputs, but also on previous inputs and/or outputs.

Main features:

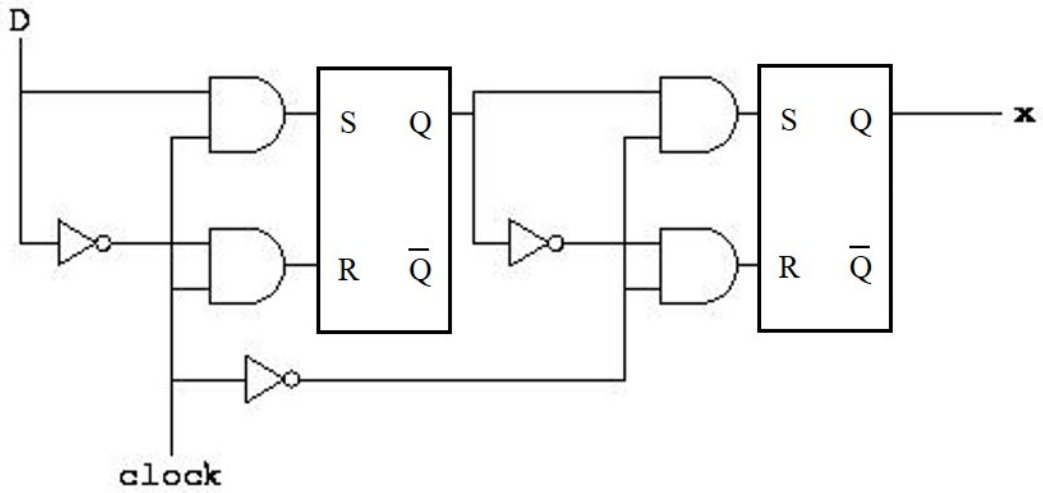
- many kinds of latches and flip-flop do exist (SR, D, T, JK, ...)
- they can be synchronous or asynchronous
- they are made of basic gates
- with feedback signals
- can not be described through a truth table

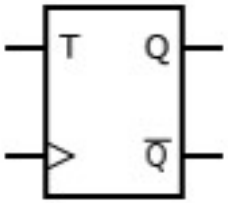
Example: asynchronous SR-latch (s = set '1', r = reset '0')



Basic memory elements: latch and flip-flop

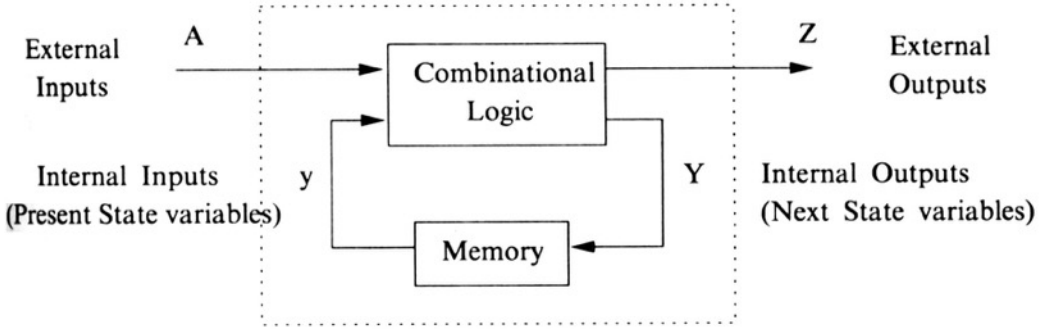
Example: master-slave D-type flip-flop





Sequential Circuits

Sequential circuit: a circuit with memory elements implementing a specific multi-input multi-output logic function



Main features:

- it has memory
- for a given logical function, different alternative combinatorial circuit can be obtained (by varying the number and/or the kind of components employed)
- can be described through a state table. If *n* is the number of inputs and *m* the number of outputs, the truth table has *2^n* rows and *n+2m* columns.

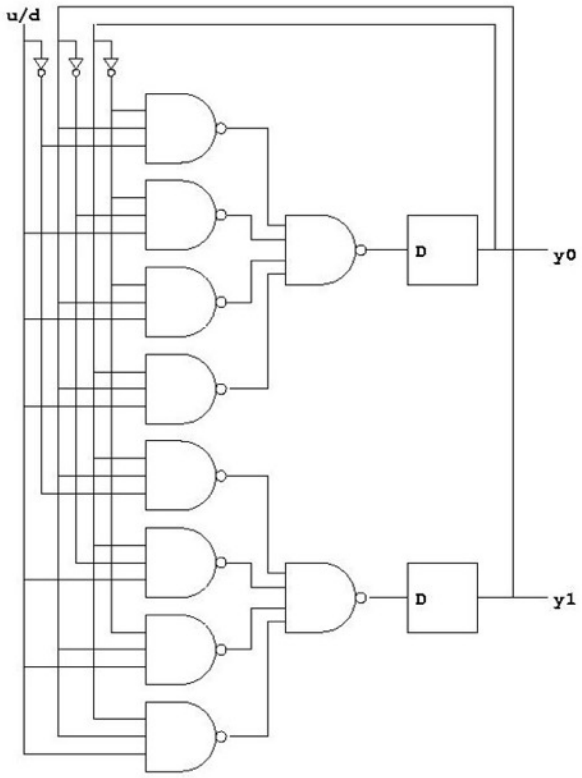
Sequential Circuits

Example: 2-bit counter with 1 input for counting direction (1 = up / 0 = down)

State table

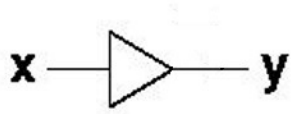
u/d	y1	y0	y1'	y0`
0	0	0	0	0
0	0	1	0	0
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	1
1	1	1	1	1

One possible implementation



Other basic elements

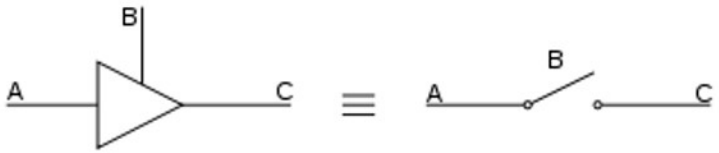
Buffer (or driver): is used for boosting the signal where needed. A buffer is a gate equivalent to a pair of consecutive inverters.



y=x

x	y
0	0
1	1

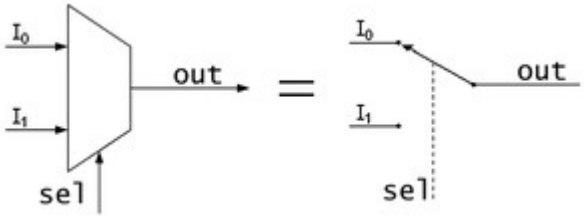
3-state buffer} is a buffer admitting a third high-impedance state. Indeed a three-state buffer can be thought of as a switch.



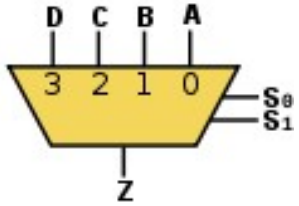
A	B	C
0	0	z
0	1	0
1	0	z
1	1	1

Other basic elements

Multiplexer: can be considered as a multiple-input, single-output switch. It selects one among the input signals and puts it into the output line.



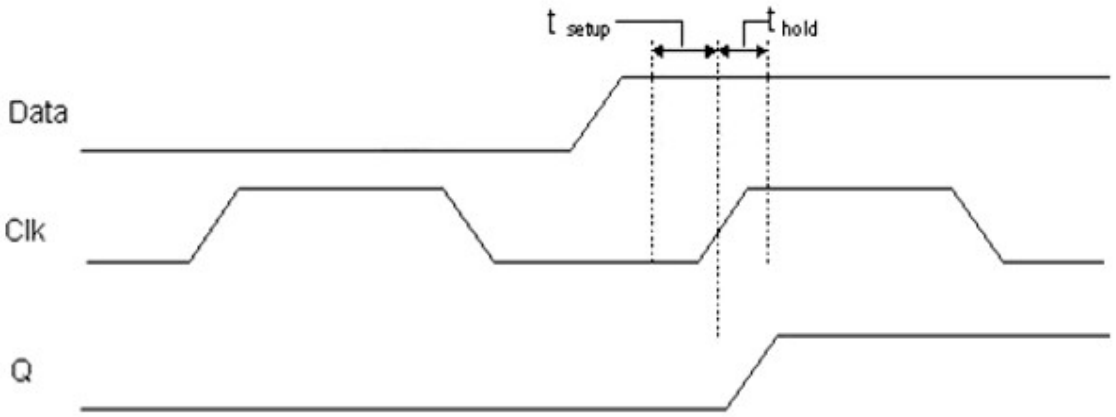
A multiplexer of *2^n* inputs has *n* select bits, used to select which input line to send out



Other basic elements

Timing diagrams: graph representing the values of a set of signals in the time domain. Its use is very common especially for components whose inputs signal must obey to some timing rules.

Example: D flip-flop state transition





Università
di Genova